ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ВНУТРІШНІХ СПРАВ

Кафедра інформаційних систем та технологій

ЛАБОРАТОРНА РОБОТА №11

з дисципліни «Об’єктно-орієнтовне програмування»

Тема: «Обробка винятків в ООП»

Виконав:

курсант 2-го курсу

ННІ 4-25-203 інф

Кожевніков К.О.

Прийняв:

Доцент кафедри ІСТ, к.т.н.,
Слободянюк О.В.

КАМ’ЯНЕЦЬ-ПОДІЛЬСЬКИЙ – 2026

ВАРІАНТ 6. Медичний центр

Завдання:

Класи для роботи: Service, Appointment, Client. Створіть AppointmentError і InvalidAppointmentStateError для вигаданих записів. Не використовуйте дані реальних осіб; перехоплюйте тільки очікувані типи винятків.

КОД РОЗВ'ЯЗКУ

```
"""
Лабораторна робота № 11
Тема: Власні класи винятків предметної області, конструкція try-except-else-
finally
Варіант 6: Медичний центр
"""

from abc import ABC, abstractmethod

# --- 1. ВЛАСНІ ВИНЯТКИ ПРЕДМЕТНОЇ ОБЛАСТІ ---
class AppointmentError(Exception):
    """Базовий виняток для всіх помилок предметної області медичного
центру."""
    pass


class InvalidAppointmentStateError(AppointmentError):
    """Виняток, що піднімається при спробі встановити невідомий або
неприпустимий статус."""
    pass


class AppointmentNotFoundError(AppointmentError):
    """Виняток, що піднімається, коли запис із вказаним номером не
знайдено."""
    pass


# Словник для реєстрації типів записів
APPOINTMENT_TYPES = {}


def register_appointment_type(cls):
    """Декоратор класу: реєструє тип запису за його унікальним атрибутом
kind."""
    if hasattr(cls, "kind"):
        APPOINTMENT_TYPES[cls.kind] = cls
    return cls
```

```python
class AuditableMixin:
    """Mixin-клас, що додає функціонал логування (аудиту) подій для запису."""

    def audit_record(self):
        """Повертає рядок для запису в журнал подій медичного центру."""
        return f"[АУДИТ] Запис №{self.number} (час: {self.time}): змінено стан
або створено подію."


class Service:
    """Модель медичної послуги."""

    def __init__(self, name, duration_min):
        self.name = name
        self.duration_min = duration_min

    def __str__(self):
        return f"Послуга «{self.name}» ({self.duration_min} хв)"

    def __repr__(self):
        return f"Service(name={self.name!r},
duration_min={self.duration_min!r})"

    def __eq__(self, other):
        if not isinstance(other, Service):
            return NotImplemented
        return self.name == other.name and self.duration_min ==
other.duration_min


class Client:
    """Модель пацієнта (клієнта) медичного центру."""

    def __init__(self, full_name, phone):
        self.full_name = full_name
        self.phone = phone

    def __str__(self):
        return f"{self.full_name} (тел: {self.phone})"

    def __repr__(self):
        return f"Client(full_name={self.full_name!r}, phone={self.phone!r})"


@register_appointment_type
class Appointment(AuditableMixin, ABC):
```

```
    """Абстрактний базовий клас запису на прийом із захистом стану через
властивість."""

    kind = "base"
    valid_statuses = {"підтверджено", "завершено", "скасовано"}
    created_count = 0

    def __init__(self, number, time, service, client, status="підтверджено"):
        self.number = number
        self.time = time
        self.service = service
        self.client = client

        self._status = "підтверджено"
        self.status = status  # Проходить через setter із перевіркою

        Appointment.created_count += 1

    @property
    def status(self):
        """Повертає поточний статус із захищеного внутрішнього поля."""
        return self._status

    @status.setter
    def status(self, value):
        """Дозволяє встановити лише допустимий статус, інакше піднімає
виняток."""
        if value not in self.valid_statuses:
            raise InvalidAppointmentStateError(f"Невідомий або неприпустимий
статус: '{value}'")
        self._status = value

    @staticmethod
    def is_valid_number(number):
        """Статичний метод: перевіряє, чи є номер додатним цілим числом."""
        return isinstance(number, int) and number > 0

    @classmethod
    def from_dict(cls, data, service, client, **kwargs):
        """Альтернативний конструктор з урахуванням специфічних полів."""
        return cls(
            number=data["number"],
            time=data["time"],
            service=service,
            client=client,
            status=data.get("status", "підтверджено"),
            **kwargs
        )
```

```python
    @abstractmethod
    def preparation_note(self):
        """Абстрактний метод підготовки до прийому."""
        pass

    def __str__(self):
        return (
            f"Запис №{self.number} | Час: {self.time} | "
            f"Пацієнт: {self.client.full_name} | Послуга: {self.service.name}
| Статус: {self.status}"
        )

    def __repr__(self):
        return (
            f"{self.__class__.__name__}(number={self.number!r},
time={self.time!r}, "
            f"service={self.service!r}, client={self.client!r},
status={self.status!r})"
        )

    def __eq__(self, other):
        if not isinstance(other, Appointment):
            return NotImplemented
        return self.number == other.number


@register_appointment_type
class Consultation(Appointment):
    """Похідний клас для консультаційних прийомів лікаря."""

    kind = "consultation"

    def __init__(self, number, time, service, client, doctor_office="101",
status="підтверджено"):
        super().__init__(number, time, service, client, status)
        self.doctor_office = doctor_office

    def preparation_note(self):
        return f"Консультація у каб. №{self.doctor_office}: очікуйте під
кабінетом, прийдіть за 10 хв."


# --- 2. СЕРВІС ДЛЯ КЕРУВАННЯ ЗАПИСАМИ ТА ОБРОБКИ ВИНЯТКІВ ---
class AppointmentService:
    """Сервіс для пошуку, створення та керування станом записів медичного
центру."""

    def __init__(self, appointments=None):
```

```
        # Зберігаємо записи у словнику для швидкого пошуку за номером {number: appointment_obj}
        self.appointments = {app.number: app for app in (appointments or [])}

    def find_by_number(self, number):
        """Шукає запис за номером або піднімає AppointmentNotFoundError."""
        try:
            return self.appointments[number]
        except KeyError as exc:
            # Зберігаємо початкову причину через raise ... from
            raise AppointmentNotFoundError(f"Запис із номером №{number} у медичному центрі не знайдено.") from exc

    def update_status(self, number, new_status):
        """Змінює статус запису за його номером."""
        appointment = self.find_by_number(number)
        # Встановлення статусу через property автоматично викличе InvalidAppointmentStateError, якщо статус невалідний
        appointment.status = new_status
        return appointment


# --- 3. ДЕМОНСТРАЦІЯ КЛІЄНТСЬКОГО СЦЕНАРІЮ ---
if __name__ == "__main__":
    s1 = Service("Консультація терапевта", 30)
    c1 = Client("Олександр Т.", "+380501112233")

    # Створюємо початковий запис
    initial_app = Consultation(101, "2026-11-08 10:00", s1, c1, doctor_office="204", status="підтверджено")
    service = AppointmentService([initial_app])

    print("=== ТЕСТ 1: Успішна зміна статусу запису ===")
    try:
        updated_app = service.update_status(101, "завершено")
    except AppointmentError as error:
        print(f"Помилка предметної області: {error}")
    else:
        print(f"Успішно оновлено! Оновлений стан: {updated_app}")
    finally:
        print("Операцію обробки статусу завершено.\n")

    print("=== ТЕСТ 2: Перехоплення InvalidAppointmentStateError (невідомий статус) ===")
    try:
        service.update_status(101, "невідомий_статус_тест")
    except InvalidAppointmentStateError as error:
        print(f"Перехоплено очікуваний статусний виняток: {error}")
    except AppointmentError as error:
```

```
        print(f"Інша помилка предметної області: {error}")
    else:
        print("Цей блок не виконається через помилку.")
    finally:
        print("Перевірку статусу завершено.\n")

    print("=== ТЕСТ 3: Перехоплення AppointmentNotFoundError (запис не
знайдено) ===")
    try:
        # Намагаємося оновити неіснуючий запис №999
        service.update_status(999, "завершено")
    except AppointmentNotFoundError as error:
        print(f"Перехоплено виняток відсутності запису: {error}")
    except AppointmentError as error:
        print(f"Інша помилка предметної області: {error}")
    else:
        print("Цей блок не виконається.")
    finally:
        print("Пошук запису завершено.")
```

РЕЗУЛЬТАТИ РОБОТИ ПРОГРАМИ

```
PS C:\Users\user\AppData\Local\Programs\Microsoft VS Code> & 'C:\Users\user\AppData\Local\Programs\Python\Python38-32\python.exe' 'c:/Users/user/Downloads/Lab11.py'
=== ТЕСТ 1: Успішна зміна статусу запису ===
Успішно оновлено! Оновлений стан: Запис №101 | Час: 2026-11-08 10:00 | Пацієнт: Олександр Т. | Послуга: Консультація терапевта | Статус: завершено
Операцію обробки статусу завершено.

=== ТЕСТ 2: Перехоплення InvalidAppointmentStateError (невідомий статус) ===
Перехоплено очікуваний статусний виняток: Невідомий або неприпустимий статус: 'невідомий_статус_тест'
Перевірку статусу завершено.

=== ТЕСТ 3: Перехоплення AppointmentNotFoundError (запис не знайдено) ===
Перехоплено виняток відсутності запису: Запис із номером №999 у медичному центрі не знайдено.
Пошук запису завершено.
```